Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Agenti AI per il CRM enterprise: automazione del testing e analisi della telemetria

Tesi di laurea triennale

Relatore

Prof. Francesco Ranzato

Laureando

Alessio Zanella

Matricola 2101079

Anno Accademico 2025-2026

Alessio Zanella: *Agenti AI per il CRM enterprise: automazione del testing e analisi della telemetria,* Tesi di laurea triennale, © Luglio 2026.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit.

— Oscar Wilde

Dedicato a ...

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa trecento ore, presso l'azienda Sia Informatica s.r.l.
Il progetto prevede l'integrazione di un sistema strutturato di testing automatizzato e telemetria all'interno dell'applicazione web *enterprise* Sia.Core, il cui core è composto da un client-side sviluppato in Angular e un server-side in .NET 9. L'approccio adottato si basa sull'utilizzo dell'intelligenza artificiale come elemento centrale del processo, con particolare riferimento a Spec-Kit, strumento che consente di definire in modo formale e strutturato le specifiche del lavoro svolto dall'AI, garantendo tracciabilità, riproducibilità e qualità nella generazione automatica del codice di test.
Per la telemetria e l'osservabilità, il progetto adotta OpenTelemetry per la raccolta distribuita di metriche, trace e log, e Grafana come piattaforma di visualizzazione e analisi, al fine di valutare le prestazioni del sistema e identificare eventuali criticità in modo proattivo.
Gli obbiettivi da raggiungere erano molteplici.
In primo luogo, è stato necessario approfondire le tecnologie richieste per lo sviluppo del progetto e comprendere l'applicativo Sia.Core, all'interno del quale la soluzione sarebbe stata integrata. In secondo luogo, si è resa necessaria l'analisi della strategia di testing più adatta al framework aziendale, insieme all'individuazione del modello di intelligenza artificiale più idoneo a supportare tale attività. Infine, il progetto è stato portato a termine attraverso l'integrazione della telemetria e l'implementazione dei test necessari al rilascio della soluzione in produzione.

"Life is really simple, but we insist on making it complicated"

— Confucius

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Francesco Ranzato, relatore della mia tesi, per l'aiuto e il sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare con affetto i miei genitori per il sostegno, il grande aiuto e per essermi stati vicini in ogni momento durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici per tutti i bellissimi anni passati insieme e le mille avventure vissute.

Padova, Luglio 2026 Alessio Zanella

Indice

1 Introduzione **1**
1.1 L'azienda . . . 1
1.2 L'idea . . . 2
1.3 Organizzazione del testo . . . 2

2 Descrizione dello stage **4**
2.1 Introduzione al progetto . . . 4
2.2 Requisiti e obiettivi . . . 4
2.2.1 Obiettivi . . . 4
2.2.2 Requisiti funzionali . . . 5
2.2.3 Requisiti non funzionali . . . 5
2.3 Analisi preventiva dei rischi . . . 6
2.4 Pianificazione . . . 7
2.4.1 Pianificazione settimanale . . . 7

3 Analisi dei requisiti **11**
3.1 Analisi del sistema . . . 11
3.2 Casi d'uso . . . 11
3.2.1 Attori . . . 11
3.2.2 Elenco casi d'uso . . . 11
3.3 Tracciamento dei requisiti . . . 12

4 Tecnologie adottate **14**
4.1 Studio e scelta delle tecnologie . . . 14
4.2 Tecnologie e strumenti . . . 14

5 Progettazione **17**
5.1 Osservabilità nei sistemi distribuiti . . . 17
5.2 Lo standard OpenTelemetry . . . 18
5.3 Architettura della telemetria . . . 18
5.3.1 Produzione dei dati di telemetria . . . 18
5.3.2 Integrazione con OpenTelemetry Collector ed ecosistema Grafana 19
5.3.3 Scelte architetturali . . . 20
5.4 Modello dei segnali e utilizzo degli attributi . . . 21
5.4.1 Modello dei log . . . 21
5.4.2 Modello delle trace . . . 21
5.4.3 Modello delle metriche . . . 21

5.4.4 Tassonomia degli attributi 22
5.4.5 Strategie di campionamento 23

6 Codifica **24**

7 Conclusioni **25**
7.1 Consuntivo finale 25
7.2 Raggiungimento degli obiettivi 25
7.3 Conoscenze acquisite 25
7.4 Valutazione personale 25

8 Roba da organizzare **26**

A Appendice A **27**

Acronimi e abbreviazioni **28**

Glossario **29**

Bibliografia **31**

Elenco delle figure

1.1 Logo Sia Informatica 1

3.1 Use Case - UC0: Scenario principale 12

4.1 Logo Angular 15
4.2 Logo .NET 15
4.3 Logo Spec-kit 16
4.4 Logo OpenTelemetry 16
4.5 Logo Grafana 16

5.1 Architettura OTel Collector 20

Elenco delle tabelle

3.1 Tabella del tracciamento dei requisti funzionali 13
3.2 Tabella del tracciamento dei requisiti qualitativi 13
3.3 Tabella del tracciamento dei requisiti di vincolo 13

5.1 Resource attributes comuni a trace, metriche e log 22
5.2 Tag del modulo Sia.Base (operazioni Create, Read, Update, Delete (CRUD)) 22
5.3 Tag degli span del modulo Sia.Auth 23
5.4 Tag degli span del modulo Sia.Grid 23

Capitolo 1

Introduzione

Il capitolo seguente descrive l'azienda ospitante e l'idea alla base del progetto di stage

1.1 L'azienda

Sia Informatica S.r.l. è una società con sede a Vigonza (PD) che, dal 1996, opera nel settore dello sviluppo software e della consulenza informatica per aziende di diversi comparti produttivi. Il *core business* dell'azienda è la progettazione e l'implementazione di soluzioni tecnologiche personalizzate, pensate per rispondere alle specifiche esigenze operative di ciascun cliente. L'offerta comprende sistemi Enterprise Resource Planning (ERP) verticali, declinati per i settori del packaging, dell'arredamento e della moda sotto il nome commerciale *Job3 ERP*, servizi web tra cui un Customer Relationship Management (CRM) (*Sales Connect*) e strumenti per la gestione logistica, soluzioni di gestione documentale (fatturazione elettronica, firma digitale e conservazione sostitutiva), nonché sistemi di integrazione avanzata come la tecnologia Radio-Frequency Identification (RFID), per la gestione intelligente della logistica e la tracciabilità aziendale, e un assistente basato sull'intelligenza artificiale denominato *Kitsune*. La filosofia aziendale si ispira al principio giapponese del Kintsugi, ovvero l'arte di valorizzare le imperfezioni trasformandole in punti di forza: in modo analogo, Sia Informatica non si limita a fornire strumenti software isolati, ma costruisce connessioni profonde tra i processi aziendali, puntando all'armonizzazione e all'ottimizzazione dei flussi operativi. Con oltre vent'anni di esperienza nel mercato, l'azienda rappresenta un partner tecnologico consolidato per le imprese del Nord-Est Italia che cercano soluzioni scalabili, integrate e su misura.

Figura 1.1: Logo Sia Informatica

1.2 L'idea

Il progetto di stage nasce dall'esigenza, da parte di Sia Informatica, di migliorare la qualità e il monitoraggio delle applicazioni *web enterprise* sviluppate internamente. In particolare, l'applicazione Sia.Core rappresenta un sistema complesso, basato su architettura *full-stack* (*Angular* per il *client* e *.NET 9* per il *server*), per il quale risulta fondamentale garantire elevati standard di affidabilità, manutenibilità e prestazioni.
In questo contesto, le attività di *testing* e di monitoraggio risultano cruciali ma, allo stesso tempo, onerose se svolte manualmente o con strumenti tradizionali. La crescente complessità del sistema e il numero sempre maggiore di componenti rendono infatti difficile mantenere una copertura di test adeguata e individuare tempestivamente eventuali anomalie o malfunzionamenti.
Per rispondere a queste criticità, il progetto propone un approccio innovativo basato sull'utilizzo dell'intelligenza artificiale, con l'obiettivo di automatizzare e migliorare sia la generazione dei test sia l'analisi del comportamento dell'applicazione in esercizio. In particolare, l'uso dello strumento *Spec-Kit* consente di definire in modo strutturato le specifiche e guidare la generazione automatica del codice di test, rendendo il processo più efficiente, riproducibile e scalabile.
Parallelamente, il progetto affronta il tema della telemetria e dell'osservabilità del sistema, attraverso l'integrazione di *OpenTelemetry* per la raccolta di metriche, *log* e *trace*, e di *Grafana* per la loro visualizzazione. Questo permette di ottenere una visione completa e in tempo reale dello stato dell'applicazione, facilitando l'individuazione di problemi e il monitoraggio delle prestazioni.

1.3 Organizzazione del testo

Il secondo capitolo descrive gli obiettivi del periodo di stage, la pianificazione del lavoro ed eventuali rischi preventivati.

Il terzo capitolo approfondisce le funzionalità del prodotto, tramite requisiti e casi d'uso implementati.

Il quarto capitolo presenta le tecnologie e gli strumenti utilizzati durante il periodo di stage e adottati per lo sviluppo del prodotto.

Il quinto capitolo affronta le caratteristiche tecniche del prodotto, con particolare riferimento alle scelte progettuali e architetturali.

Il sesto capitolo espone ...

Nel settimo capitolo descrive ...

Riguardo la stesura del testo, relativamente al documento sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato alla fine del presente documento;

- per la prima occorrenza dei termini riportati nel glossario viene utilizzata la nomenclatura "parola(abbreviazione)", mentre per ogni successiva occorrenza verrà utilizzata solamente l'abbreviazione di tale termine;
- i termini in lingua straniera e facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

Capitolo 2

Descrizione dello stage

Il presente capitolo descrive gli obiettivi del progetto di stage, la pianificazione delle attività in termini di ore di lavoro e l'analisi preventiva dei principali rischi che potrebbero insorgere nel corso del suo svolgimento.

2.1 Introduzione al progetto

Nel contesto dello sviluppo software moderno, le applicazioni tendono ad assumere un'architettura sempre più distribuita e complessa. La capacità di monitorare il comportamento di tali sistemi in tempo reale, diagnosticare anomalie e analizzare le prestazioni costituisce un requisito fondamentale per garantire la qualità del servizio.

Il concetto di *osservabilità* (*observability*) si riferisce alla possibilità di inferire lo stato interno di un sistema a partire dai dati che esso emette verso l'esterno. In ambito applicativo, i principali vettori di osservabilità sono tre: le *trace distribuite*, che tracciano il percorso di una richiesta attraverso il sistema; le *metriche*, che forniscono misure aggregate nel tempo; i *log*, che registrano gli eventi discreti generati dall'applicazione. La convergenza di questi tre segnali in un'unica piattaforma è oggi indicata come *observability stack* o, più informalmente, con l'acronimo *MELT* (Metrics, Events, Logs, Traces).

Il progetto *Sia.Web* ha richiesto la progettazione e l'implementazione di un sistema di telemetria completo, in grado di raccogliere e correlare questi tre tipi di segnale per offrire una visione unificata del comportamento delle applicazioni in produzione.

2.2 Requisiti e obiettivi

2.2.1 Obiettivi

1. Analizzare i requisiti di osservabilità dell'ecosistema *Sia.Web* e identificare le soluzioni tecnologiche adeguate.
2. Progettare un'architettura di telemetria basata sullo standard OpenTelemetry (OTel), integrando il framework con le tecnologie .NET esistenti.
3. Definire le strategie di campionamento delle trace e di gestione delle metriche in ottica di scalabilità verso ambienti di produzione.

4. Implementare gli strumenti di instrumentazione per i moduli applicativi principali (autenticazione, accesso ai dati, griglia).
5. Configurare la piattaforma di visualizzazione Grafana e i relativi backend di storage.

2.2.2 Requisiti funzionali

RF1. Il sistema deve tracciare il percorso completo di ogni richiesta HTTP in ingresso, incluse le operazioni di accesso al database.

RF2. Le operazioni CRUD verso il database devono essere riconoscibili tramite attributi che espongono il metodo e la repository da cui è stata scatenata la richiesta.

RF3. Il sistema deve esporre metriche operative per i moduli di autenticazione, accesso ai dati e gestione della griglia.

RF4. I log devono essere correlabili alle trace tramite `TraceId` e `SpanId`.

RF5. Il sistema deve essere attivabile e disattivabile tramite configurazione, senza richiedere modifiche al codice.

RF6. Deve essere possibile configurare indipendentemente il campionamento per ciascuna sorgente di trace.

RF7. Deve essere possibile visualizzare i valori di trace e metriche raggruppate in dashboard Grafana.

RF8. Le dashboard di Grafana devono raccogliere i dati da più servizi e installazioni diverse di ciascun cliente, e permettere di filtrare per gli stessi.

RF9. Sviluppo di Claude skill per permettere la propagazione della telemetria in tutto il sistema.

2.2.3 Requisiti non funzionali

RNF1. Identificazione delle tecnologie più adatte allo sviluppo del progetto.

RNF2. Identificazione dei moduli più importanti da tracciare tramite telemetria.

RNF3. La telemetria non deve degradare sensibilmente le prestazioni in produzione.

RNF4. Le metriche devono essere progettate per non generare un numero eccessivo di serie temporali in Prometheus.

RNF5. I segnali prodotti devono essere compatibili con qualsiasi backend OTel-compatibile.

RNF6. Le impostazioni devono essere modificabili per ambiente (sviluppo, staging, produzione) tramite appsettings.

2.3 Analisi preventiva dei rischi

Durante la fase iniziale di analisi, sono stati individuati i principali rischi che sarebbero potuti emergere nel corso dello stage. Per ciascun rischio identificato, è stata definita una possibile strategia di mitigazione, al fine di ridurne l'impatto e garantire il corretto avanzamento delle attività previste.

1. Tecnologie sconosciute
Descrizione: alcune delle tecnologie previste nel progetto risultano totalmente o parzialmente sconosciute, con il rischio di rallentare le attività nelle fasi iniziali dello stage.
Soluzione: pianificazione di un'attività strutturata di studio e approfondimento, attraverso l'utilizzo di documentazione ufficiale, tutorial online, corsi forniti dall'azienda e lo sviluppo di piccoli progetti di consolidamento, al fine di acquisire rapidamente le competenze necessarie.

2. Gestione del tempo
Descrizione: la pianificazione delle attività potrebbe risultare non accurata, soprattutto a causa della presenza di tecnologie nuove e della componente sperimentale legata all'uso dell'intelligenza artificiale, con il rischio di ritardi rispetto alle scadenze previste.
Soluzione: adottare il modello agile SCRUM per definire obiettivi intermedi settimanali, monitoraggio continuo dello stato di avanzamento e revisione periodica della pianificazione, in modo da adattare le attività in base alle difficoltà riscontrate.

3. Allucinazioni Large Language Model (LLM)
Descrizione: i modelli di intelligenza artificiale utilizzati per la generazione automatica di test possono produrre risultati non corretti, incoerenti o non aderenti alle specifiche (fenomeno noto come "allucinazione"), compromettendo l'affidabilità del codice generato.
Soluzione: adozione di tecniche di verifica e validazione dei risultati prodotti, affiancate da un'attenta progettazione dei *prompt* e dall'utilizzo di specifiche strutturate. Inoltre, iterazioni successive e revisioni manuali consentono di migliorare progressivamente la qualità dell'output generato.

4. Integrazione tra tecnologie
Descrizione: l'integrazione tra strumenti diversi, quali Spec-Kit, OpenTelemetry *e* Grafana, può presentare difficoltà legate a compatibilità e interfacciabilità, con il rischio di rallentare lo sviluppo o compromettere il corretto flusso dei dati.
Soluzione: utilizzo di un approccio incrementale all'integrazione, partendo da configurazioni di base e aumentando progressivamente la complessità. Consultazione della documentazione, utilizzo di ambienti di sviluppo locali per test e validazione, e confronto con il team aziendale per la risoluzione di eventuali criticità.

5. Qualità dei test generati automaticamente
Descrizione: i test generati tramite modelli di intelligenza artifutilizzo di un approccio incrementale all'integrazione, partendo da coiciale potrebbero risultare incompleti, non corretti o con copertura insufficiente, compromettendo l'efficacia del

processo di *testing*.
Soluzione: introduzione di attività di validazione e revisione dei test generati, utilizzo di metriche di copertura del codice e iterazione sui *prompt* per migliorare progressivamente la qualità dei risultati.

6. Differenze di comportamento tra ambiente di sviluppo e produzione
Descrizione: il sistema potrebbe presentare comportamenti differenti tra ambiente di sviluppo, test e produzione, a causa di differenze nella configurazione, nei dati o nell'infrastruttura. Ciò può portare a malfunzionamenti non rilevati durante le fasi di sviluppo e *testing*.
Soluzione: utilizzo di ambienti di *staging* il più possibile allineati alla produzione, esecuzione di test approfonditi prima del rilascio e monitoraggio continuo tramite strumenti di telemetria, al fine di individuare e risolvere tempestivamente eventuali anomalie.

7. Overfitting dei prompt per modelli LLM
Descrizione: i *prompt* progettati per la generazione automatica dei test potrebbero risultare troppo specifici rispetto ai casi inizialmente considerati, adattandosi eccessivamente al codice analizzato nelle prime fasi. Questo fenomeno può portare a una ridotta capacità di generalizzazione, con conseguente inefficacia nella generazione di test corretti o completi per nuove porzioni di codice introdotte successivamente.
Soluzione: progettazione dei *prompt* in modo generale e modulare, evitando eccessiva specializzazione su casi specifici. Introduzione di test su scenari diversi e codice non visto durante le fasi iniziali, accompagnata da iterazioni e revisioni periodiche dei *prompt*, al fine di migliorarne la capacità di generalizzazione.

2.4 Pianificazione

L'attività di stage è stata pianificata su una durata complessiva di 8 settimane, per un totale di 300 ore.
La pianificazione è stata strutturata in modo progressivo, suddividendo il lavoro in fasi settimanali, ciascuna con obiettivi specifici e attività mirate, al fine di garantire un apprendimento graduale e un'integrazione efficace delle tecnologie coinvolte.

2.4.1 Pianificazione settimanale

2.4.1.1 Settimane 1 e 2: Studio dell'architettura del sistema

Le prime due settimane sono state dedicate all'analisi dell'architettura dell'applicazione Sia.Core, con particolare attenzione alla separazione tra componente *client-side* e *server-side*.

- Analisi della struttura del progetto *Angular*: moduli, componenti, servizi, *routing*
- Studio delle convenzioni e pattern architetturali adottati (*lazy loading, state management*)
- Revisione del codice esistente con supporto del team di sviluppo

- Analisi della struttura del progetto *.NET 9*: *layer, Application Program Interface (API), middleware, dependency injection*
- Studio dei pattern architetturali utilizzati (*Clean Architecture, CQRS, repository pattern*)
- Comprensione dei flussi di dati tra *client* e *server*: *API REST*, contratti, autenticazione
- Mappatura dei componenti critici candidati all'integrazione di test e telemetria

2.4.1.2 Settimana 3: Studio dell'architettura del sistema

La terza settimana è stata dedicata all'apprendimento degli strumenti chiave del progetto: *Spec-Kit*, *OpenTelemetry* e *Grafana*.

- *Spec-Kit*: studio della documentazione, comprensione del modello di specifica *AI-driven*, esercitazioni pratiche
- *OpenTelemetry*: studio dei concetti fondamentali (*traces, metrics, logs*), integrazione con *Angular* e *.NET*
- *Grafana*: configurazione istanza locale, connessione alle sorgenti dati *OpenTelemetry*, dashboard di base
- Realizzazione di un ambiente di sviluppo locale integrato con tutti e tre gli strumenti

2.4.1.3 Settimana 4: Strategia di testing e selezione del modello AI

Durante la quarta settimana è stata definita la strategia di *testing* da adottare, identificando le principali tipologie di test (unitari, di integrazione, *end-to-end* e di performance).
Parallelamente, è stata effettuata un'analisi comparativa tra diversi modelli di intelligenza artificiale, con l'obiettivo di selezionare quello più adatto alla generazione automatica di test in integrazione con *Spec-Kit*.

- Identificazione delle tipologie di test: *unit test, integration test, E2E test, performance test*
- Valutazione comparativa dei modelli Artificial Intelligence (AI) disponibili (*GPT-4o, Claude, Gemini*) per qualità e integrazione con *Spec-Kit*
- Selezione e motivazione del modello AI adottato
- Stesura del documento di strategia condiviso con il team

2.4.1.4 Settimana 5: Prompt engineering e prima generazione di test

La quinta settimana è stata dedicata allo studio delle tecniche di *prompt engineering* e alla prima generazione di test tramite intelligenza artificiale.

- Studio delle tecniche di *prompt engineering*: *chain-of-thought, few-shot, context injection*
- Definizione delle specifiche tramite *Spec-Kit* per i componenti individuati nella settimana 4
- Stesura dei *prompt* per la generazione di test lato *client* (*Angular*) e lato *server* (*.NET 9*)
- Prima esecuzione dei *prompt* sul modello AI selezionato
- Raccolta, analisi e valutazione del codice generato

2.4.1.5 Settimana 6: Iterazione e integrazione della telemetria

Nella sesta settimana il lavoro si è concentrato sul miglioramento iterativo dei test generati e sull'integrazione della telemetria nel sistema.

- Analisi dei risultati: identificazione di test mancanti, errati o incompleti
- Raffinamento dei *prompt* e nuove iterazioni di generazione
- Integrazione di *OpenTelemetry* nei componenti server-side: trace e metrics sulle API principali
- Prima configurazione delle dashboard *Grafana* per la visualizzazione dei dati
- Esecuzione dei test e verifica dei risultati su ambiente di sviluppo
- Documentazione delle iterazioni e delle decisioni prese

2.4.1.6 Settimana 7: Iterazione e integrazione della telemetria

La settima settimana ha previsto l'esecuzione di una campagna completa di test e la validazione dell'intero sistema integrato.

- Esecuzione della suite completa di test su ambiente di *staging*
- Analisi della copertura del codice e identificazione delle aree scoperte
- Verifica della telemetria *end-to-end*: dall'evento applicativo alla dashboard *Grafana*
- Ottimizzazione delle dashboard *Grafana*: alert, soglie, visualizzazioni avanzate
- Correzione dei test falliti e ulteriore iterazione sui *prompt* se necessario
- Stesura del report di qualità intermedio

2.4.1.7 Settimana 8: Iterazione e integrazione della telemetria

L'ultima settimana è stata dedicata alla messa in produzione della soluzione e alla redazione della documentazione finale.

- Preparazione e revisione finale dell'ambiente di produzione
- Deploy della configurazione *OpenTelemetry* e *Grafana* in produzione
- Stesura della documentazione tecnica finale (architettura, *prompt*, modello AI, risultati)
- Stesura della relazione di stage con analisi critica del lavoro svolto
- Presentazione finale dei risultati al team aziendale

Capitolo 3

Analisi dei requisiti

Il presente capitolo descrive le funzionalità del prodotto, formalizzate sotto forma di requisiti di progetto, e relativi casi d'uso associati.

3.1 Analisi del sistema

Il prodotto sviluppato durante il periodo di stage, mette a disposizione dello sviluppatore un sistema che permetta la creazione automatizzata di test e il monitoraggio dello stato del sistema Sia.Core tramite dashboard. Continua con descrizione tecnica...

3.2 Casi d'uso

La presente sezione illustra i casi d'uso analizzati, corredati dai relativi diagrammi. I diagrammi dei casi d'uso (in inglese *Use Case Diagram*) costituiscono una tipologia di diagrammi Unified Modeling Language (UML) finalizzati alla rappresentazione delle funzionalità e dei servizi offerti da un sistema, dal punto di vista degli attori che interagiscono con esso.

3.2.1 Attori

Gli attori nei casi d'uso rappresentano entità esterne al sistema, quali utenti o altri servizi, che interagiscono con esso. In merito al sistema sviluppato si possono individuare come attori:

- **Sviluppatore**: è l'attore principale che utilizza il sistema
- **LLM**: descrizione
- **Sia.Core**: descrizione

3.2.2 Elenco casi d'uso

- Generazione test (sarà necessario specificare un determinato componente/classe per i quali generare test)
- Singole visualizzazioni nella dashboard Grafana

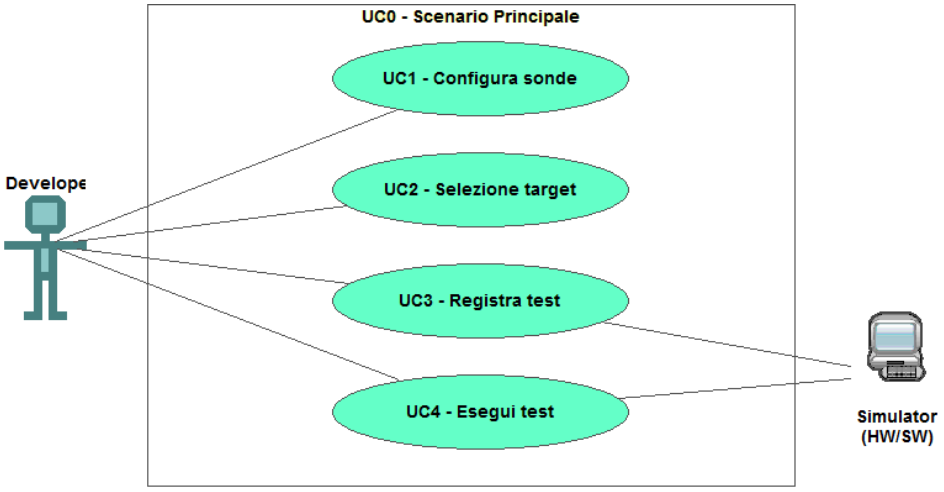

Figura 3.1: Use Case - UC0: Scenario principale

UC0: Scenario principale

Attori Principali: Sviluppatore applicativi.

Precondizioni: Lo sviluppatore è entrato nel plug-in di simulazione all'interno dell'IDE.

Descrizione: La finestra di simulazione mette a disposizione i comandi per configurare, registrare o eseguire un test.

Postcondizioni: Il sistema è pronto per permettere una nuova interazione.

3.3 Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e degli use case effettuata sul progetto è stata stilata la tabella che traccia i requisiti in rapporto agli use case.
Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.
Il codice dei requisiti è così strutturato R(F/Q/V)(N/D/O) dove:

R = requisito

F = funzionale

Q = qualitativo

V = di vincolo

N = obbligatorio (necessario)

D = desiderabile

Z = opzionale

Nelle tabelle **??**, **??** e **??** sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

Tabella 3.1: Tabella del tracciamento dei requisti funzionali

Requisito	Descrizione	Use Case
RFN-1	L'interfaccia permette di configurare il tipo di sonde del test	UC1

Tabella 3.2: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Use Case
RQD-1	Le prestazioni del simulatore hardware deve garantire la giusta esecuzione dei test e non la generazione di falsi negativi	-

- Il sistema deve permettere la generazione automatizzata di test
-

Tabella 3.3: Tabella del tracciamento dei requisiti di vincolo

Requisito	Descrizione	Use Case
RVO-1	La libreria per l'esecuzione dei test automatici deve essere riutilizzabile	-

Capitolo 4

Tecnologie adottate

Il presente capitolo descrive le tecnologie adottate per lo sviluppo del prodotto e ne illustra le principali caratteristiche.

4.1 Studio e scelta delle tecnologie

Una parte fondamentale dello stage è stata dedicata all'attività di ricerca, analisi e selezione delle tecnologie più adeguate per lo sviluppo del progetto. Questa fase iniziale ha rivestito un ruolo cruciale, in quanto le scelte tecnologiche influenzano in modo significativo sia la qualità del prodotto finale sia la sostenibilità dello sviluppo nel lungo periodo. Il processo di selezione è iniziato con una prima fase di scrematura delle soluzioni disponibili sul mercato, effettuata sulla base dei requisiti e delle esigenze espresse dall'azienda. In particolare, sono stati considerati diversi criteri di valutazione, tra cui le funzionalità offerte dalle tecnologie, i costi associati al loro utilizzo e il grado di integrazione con il sistema già esistente. Successivamente, le tecnologie ritenute più promettenti sono state sottoposte a una fase di sperimentazione pratica attraverso lo sviluppo di Proof of Concept (PoC). Questa attività ha permesso di valutare concretamente le prestazioni, l'efficacia e la compatibilità delle soluzioni selezionate, fornendo elementi utili per una scelta consapevole e motivata delle tecnologie da adottare nel progetto.

4.2 Tecnologie e strumenti

Di seguito è fornita una descrizione generale delle tecnologie e degli strumenti utilizzati per lo sviluppo del progetto.

Angular

Framework open-source per lo sviluppo di applicazioni *web front-end*, basato su *TypeScript* e mantenuto da *Google*.

- **Architettura a componenti**: *Angular* promuove una struttura modulare basata su componenti riutilizzabili, facilitando la manutenzione e la scalabilità dell'applicazione.

- **Strumenti integrati**: offre soluzioni native per *routing*, gestione dello stato, form e comunicazione con API, riducendo la necessità di librerie esterne.
- **Supporto alle SPA**: è progettato per lo sviluppo di Single Page Application (SPA), garantendo un'esperienza utente fluida e dinamica grazie all'aggiornamento asincrono dei contenuti.

Figura 4.1: Logo Angular

.NET

Piattaforma di sviluppo *software open-source* e multipiattaforma sviluppata da *Microsoft*, utilizzata per la creazione di applicazioni web, desktop, mobile e cloud.

- **Elevate prestazioni e scalabilità**: .NET garantisce alte performance e una gestione efficiente delle risorse, risultando adatto ad applicazioni *enterprise* e ad alto carico.
- **Architettura strutturata e flessibile**: supporta *pattern* architetturali come *Clean Architecture* e *Dependency Injection*, favorendo la manutenibilità e l'organizzazione del codice.
- **Ampio ecosistema e integrazione**: mette a disposizione numerose librerie e strumenti per lo sviluppo di API, gestione dei dati, sicurezza e integrazione con servizi esterni.

Figura 4.2: Logo .NET

Spec-kit

Strumento utilizzato per definire in modo strutturato e formale le specifiche da fornire a modelli di intelligenza artificiale per la generazione automatica di codice. Consente di rendere il processo *AI-driven* più tracciabile, riproducibile e controllabile, migliorando la qualità dell'output generato, in particolare nel contesto della generazione di test software.

Figura 4.3: Logo Spec-kit

OpenTelemetry

Insieme di strumenti, API e librerie *open-source* per la raccolta, generazione ed esportazione di dati di telemetria, quali metriche, *log* e *trace*, provenienti da applicazioni software. È utilizzato per implementare sistemi di osservabilità distribuita, permettendo il monitoraggio del comportamento e delle prestazioni di un sistema in tempo reale.

Figura 4.4: Logo OpenTelemetry

Grafana

Piattaforma *open-source* per la visualizzazione e l'analisi di dati, utilizzata principalmente per il monitoraggio di sistemi e applicazioni. Consente di creare dashboard interattive a partire da diverse fonti di dati (come *OpenTelemetry*), facilitando l'interpretazione delle metriche e l'individuazione di anomalie o criticità.

Figura 4.5: Logo Grafana

Capitolo 5

Progettazione

In questo capitolo si esporranno le caratteristiche tecniche della soluzione prodotta, descrivendo la progettazione e l'architettura adottata

5.1 Osservabilità nei sistemi distribuiti

L'osservabilità è la proprietà di un sistema software che permette di comprenderne lo stato interno esaminando esclusivamente i dati prodotti in output. Nel contesto delle applicazioni web distribuite, questa proprietà è resa possibile dalla produzione sistematica di tre categorie di segnali:

- **Logs**: rappresentano eventi discreti generati dal sistema durante la sua esecuzione. Contengono informazioni dettagliate su operazioni svolte, errori, stati e messaggi diagnostici, risultando fondamentali per attività di debugging e analisi puntuale di anomalie.
- **Traces**: descrivono il flusso di esecuzione di una richiesta all'interno del sistema, tracciandone il percorso attraverso i vari componenti e servizi. Sono particolarmente utili in architetture distribuite, in quanto permettono di analizzare le dipendenze tra servizi e individuare eventuali colli di bottiglia o punti di errore.
- **Metriche**: rappresentano misurazioni numeriche aggregate nel tempo, utilizzate per monitorare lo stato e le prestazioni del sistema. Si distinguono in:
  - *Metriche di performance*: raccolte automaticamente tramite strumenti come OpenTelemetry, includono informazioni quali tempi di risposta, utilizzo delle risorse, throughput e latenza, utili per valutare le prestazioni del sistema.
  - *Metriche di business*: definite a livello applicativo, riguardano aspetti funzionali del dominio, come numero di operazioni effettuate, utenti attivi o eventi specifici, e consentono di monitorare l'utilizzo e il valore del sistema dal punto di vista aziendale.

La forza di un sistema di osservabilità moderno non risiede nell'utilizzo isolato di questi segnali, bensì nella loro correlazione: poter passare da una metrica anomala alla trace corrispondente, e da questa ai log emessi durante quella specifica richiesta, costituisce uno strumento diagnostico di grande efficacia.

5.2 Lo standard OpenTelemetry

OTel è un framework open-source, oggi standard de facto nel settore, che fornisce un modello unificato e vendor-neutral per la produzione, la raccolta e l'esportazione di segnali di osservabilità. È il risultato della fusione tra i progetti *OpenCensus* e *OpenTracing*, avvenuta nel 2019 sotto l'egida della Cloud Native Computing Foundation (CNCF).

I principi fondamentali di OTel rilevanti per questo progetto sono:

- **Separazione tra produzione ed esportazione.** Le API applicative sono indipendenti dal backend di storage scelto: lo stesso codice può inviare dati a Grafana, Jaeger, Zipkin o qualsiasi altro sistema compatibile.
- **Instrumentazione automatica.** Per i framework più diffusi (ASP.NET Core, HttpClient, SqlClient), OTel fornisce librerie che non richiedono modifiche al codice applicativo.
- **Estensibilità.** È possibile aggiungere attributi custom, definire metriche applicative proprie e configurare sampler e processor personalizzati.

5.3 Architettura della telemetria

Il sistema di telemetria sviluppato si basa su una distinzione tra due macro-componenti principali: la **produzione dei dati** lato applicazione e la **raccolta e visualizzazione** lato infrastruttura.

L'applicazione backend, sviluppata in ambiente *.NET*, utilizza lo standard open-source OpenTelemetry per generare i tre principali segnali di osservabilità: *trace*, *metriche* e *log*. Questi dati vengono successivamente inviati a un'infrastruttura centralizzata basata sullo stack Grafana, che ne consente l'analisi e la visualizzazione.

5.3.1 Produzione dei dati di telemetria

La generazione dei dati avviene direttamente all'interno dell'applicazione backend tramite diverse tecnologie integrate.

Le **trace** vengono prodotte utilizzando OpenTelemetry, che si appoggia alle API native di .NET (`ActivitySource` e `Activity`) per rappresentare le operazioni eseguite dal sistema sotto forma di *span*. Ogni richiesta viene quindi tracciata lungo tutto il suo ciclo di vita, includendo chiamate HTTP, operazioni sui database e logica applicativa.

Le **metriche** vengono generate attraverso due approcci distinti:

- metriche di *performance*, raccolte automaticamente da OpenTelemetry (ad esempio durata delle richieste HTTP o utilizzo delle risorse);

- metriche di *business*, definite manualmente tramite le API `Meter` di .NET, per monitorare aspetti specifici del dominio applicativo.

I **log** vengono invece gestiti tramite la libreria Serilog, che consente la produzione di log strutturati e la loro esportazione verso l'infrastruttura di osservabilità. Grazie all'integrazione con OpenTelemetry, i log risultano automaticamente correlati alle trace tramite identificativi condivisi (*TraceId* e *SpanId*).

5.3.2 Integrazione con OpenTelemetry Collector ed ecosistema Grafana

Per la gestione e la visualizzazione dei dati di telemetria è stata adottata un'architettura basata su OpenTelemetry Collector e sullo stack Grafana (LGTM: Loki, Grafana, Tempo e Prometheus). I dati di telemetria generati dall'applicazione backend, vengono inizialmente prodotti tramite le librerie OpenTelemetry, .NET Meters e Serilog, e poi vengono esportati in formato compatibile OpenTelemetry. Tutti questi dati vengono inviati a un componente intermedio, l'*OpenTelemetry Collector*, tramite protocollo OTLP (OpenTelemetry Protocol). Il Collector svolge un ruolo centrale nell'architettura di osservabilità, fungendo da punto di raccolta e smistamento dei dati. Le sue principali responsabilità includono:

- **Ricezione dei dati**: acquisizione di trace, metriche e log dall'applicazione tramite endpoint OTLP;
- **Elaborazione**: possibilità di applicare trasformazioni, filtri o arricchimenti ai dati raccolti;
- **Esportazione**: inoltro dei dati verso i sistemi di destinazione tramite specifici *exporter*.

Nel progetto, il Collector è configurato per instradare i diversi tipi di dati verso componenti distinti dello stack Grafana:

- le **trace** vengono esportate verso Tempo, permettendo l'analisi dei flussi di esecuzione e delle dipendenze tra servizi;
- le **metriche** vengono inviate a Prometheus, rendendo possibile il monitoraggio delle prestazioni e la creazione di dashboard;
- i **log** vengono esportati verso Loki, consentendo la consultazione centralizzata e la correlazione con trace e metriche .

Grafana rappresenta il punto di accesso unificato a queste informazioni, offrendo strumenti per la visualizzazione e l'analisi dei dati di telemetria. In particolare, permette di costruire dashboard personalizzate basate su metriche, esplorare i log e navigare le trace, abilitando una visione completa e correlata del comportamento del sistema. Questa architettura consente di separare la raccolta dei dati dalla loro visualizzazione, migliorando la scalabilità e la flessibilità del sistema di monitoraggio, oltre a facilitare eventuali estensioni future.

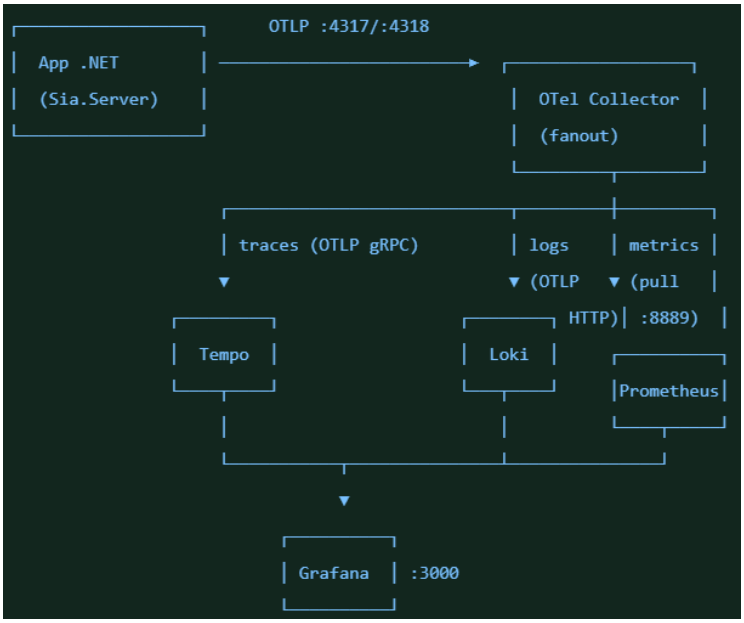

Figura 5.1: Architettura OTel Collector

5.3.3 Scelte architetturali

5.3.3.1 Adozione di OpenTelemetry

La scelta di OTel, come richiesto dal requisito RNF5, permette flessibilità nella scelta del backend di visualizzazione. Le alternative considerate, soluzioni proprietarie come Azure Application Insights o Datadog APM, avrebbero introdotto un accoppiamento stretto con un singolo fornitore, limitando la flessibilità nella scelta del backend e rendendo difficoltosa la migrazione futura.

5.3.3.2 Architettura opt-in

L'intero sistema è progettato come *opt-in*: la telemetria è attiva solo se `OpenTelemetry:Enabled = true` in appsettings, rispettando così il requisito RNF6. Questo consente di abilitarla esclusivamente negli ambienti che la richiedono, evitando overhead in sviluppo locale.

5.3.3.3 Centralizzazione della configurazione

Tutta la configurazione OTel è centralizzata in `BaseApplicationBuilder`, componente condiviso da tutti i prodotti *Sia.Web*. I singoli moduli si limitano a registrare il proprio `ActivitySource` e `Meter` nel provider già costruito, senza conoscere i dettagli del pipeline.

5.3.3.4 Campionamento a due livelli

Una limitazione del Software Development Kit (SDK) OTel .NET, è l'impossibilità di identificare la source dello span al momento dell'invocazione del sampler, ciò ha richiesto la progettazione di un meccanismo a due livelli:

1. Un *sampler globale* (`GlobalRatioParentBasedSampler`) decide se campionare le root span applicando il ratio globale.
2. Un *processor* (`PerSourceFilterProcessor`), attivo nell'hook `OnStart`, applica gli override per-source dopo che l'Activity è inizializzata e la source è identificabile con certezza.

5.4 Modello dei segnali e utilizzo degli attributi

5.4.1 Modello dei log

I log sono prodotti da Serilog con l'interfaccia `ILogger<T>` e arricchiti automaticamente con `TraceId` e `SpanId` quando emessi nel contesto di una richiesta tracciata. Questo abilita la navigazione bidirezionale tra log e trace in Grafana.

5.4.2 Modello delle trace

Una *trace* è l'insieme completo di un'operazione end-to-end, identificata da un unico `trace_id`. È composta da una gerarchia di *span*, ciascuno con un proprio `span_id` e, ad eccezione della root, un `parent_span_id`.

Nel sistema, le trace sono prodotte da tre tipologie di sorgenti: instrumentazione automatica HTTP (richieste in ingresso via AspNetCore, chiamate in uscita via HttpClient); instrumentazione automatica database (query SQL Server via SqlClient, query PostgreSQL via Npgsql); instrumentazione manuale dei moduli Auth, Base e Grid tramite le classi `Sia[NomeModulo]Diagnostics`.

5.4.3 Modello delle metriche

Sono impiegati due tipi di strumento metrico:

Counter Contatore monotono crescente per eventi cumulativi (es. login riusciti, query eseguite). In Prometheus acquisisce il suffisso `_total`.

Histogram Distribuzione statistica per latenze e percentili (p50, p95, p99). In Prometheus genera tre serie: `_bucket`, `_sum`, `_count`.

La progettazione delle metriche ha seguito due criteri guida. Il primo è la *bassa cardinalità*: i tag con alta variabilità (es. identificatori numerici di record) sono applicati solo agli span, mai alle metriche, per evitare un'esplosione del numero di serie temporali in Prometheus. Il secondo è la *pre-aggregazione*: le metriche vengono aggregate lato applicazione prima dell'esportazione, con overhead nettamente inferiore rispetto alla raccolta di trace complete.

5.4.4 Tassonomia degli attributi

5.4.4.1 Resource attributes (comuni a tutti i segnali)

Attributi presenti in tutte le metriche, trace e log di tutti i moduli.

Tabella 5.1: Resource attributes comuni a trace, metriche e log

Attributo	Chiave appsettings	Scopo
`service.name`	`ServiceName`	Identifica l'installazione
`sia.product_type`	`ProductType`	Linea di prodotto
`sia.customer_name`	`CustomerName`	Nome del cliente

5.4.4.2 Attributi del modulo Sia.Base

Tutti i tag utilizzati nel modulo Sia.Base, modulo che si occupa delle chiamate CRUD al database, sono i medesimi sia per le trace che per le metriche.

Tabella 5.2: Tag del modulo Sia.Base (operazioni CRUD)

Tag	Significato	Valori esempio
`sia.operation`	Tipo di operazione CRUD	`insert`, `update`, `delete`, `find_one`, `find_many`, `relaz_check`, `custom`
`sia.method`	Metodo infrastrutturale	`InsertSiaEntity`, `UpdateSiaEntity`, `FindSiaEntities`
`sia.caller_method`	Metodo business chiamante	`CreateOrder`, `SaveInvoice`
`sia.entity`	Tipo C# dell'entità	`OrderEntity`, `UserEntity`
`sia.repository`	Classe repository	`OrdersRepository`, `CatalogRepository`
`sia.relaz.kind`	Tipo di controllo relazione	`relaz`, `view-relaz`
`exception.type`	Tipo eccezione (solo errori)	`SqlException`, `TimeoutException`

5.4.4.3 Attributi del modulo Sia.Auth

I tag nelle metriche del modulo Sia.Auth sono a bassa cardinalità, di fatto solo la metrica `sia.auth.login.failure` contiene il tag `reason` che decrive il motivo dell'errore.
Mentre le trace, contengono i seguenti attributi:

Tabella 5.3: Tag degli span del modulo Sia.Auth

Tag	Valori possibili
`auth.method`	`credentials`
`auth.result`	`success`, `unauthorized`, `error`, `bad_request`, `two_factor_required`
`auth.two_factor_required`	`true` (solo se 2FA richiesto)
`auth.two_factor_type`	`email`, `totp`

5.4.4.4 Attributi del modulo Sia.Grid

Le metriche, come le trace del modulo Sia.Grid, contengono tutte l'attributo `componentId`.

Tabella 5.4: Tag degli span del modulo Sia.Grid

Span	Tag
`sia.grid.page`	`componentId`, `hasSpName`
`sia.grid.summary`	`componentId`
`sia.grid.massive_delete`	`componentId`, `spName`

5.4.5 Strategie di campionamento

La strategia di campionamento deve bilanciare visibilità diagnostica e costi di storage. I criteri adottati sono:

- **Campionamento globale ridotto:** In produzione, un ratio del 10% è sufficiente per una rappresentazione statistica affidabile.
- **Campionamento pieno per eventi critici:** I moduli con impatto sulla sicurezza devono essere campionati al 100% per garantire tracciabilità completa.
- **Soppressione selettiva del rumore:** Sorgenti ad alto volume e basso valore diagnostico possono essere campionate a ratio bassissimi o soppresse tramite filtri.
- **Cardinalità controllata:** I tag delle metriche devono escludere identificatori ad alta variabilità per mantenere il numero di serie temporali entro limiti accettabili.

Capitolo 6

Codifica

Attributi di metriche e trace

Capitolo 7

Conclusioni

7.1 Consuntivo finale

7.2 Raggiungimento degli obiettivi

7.3 Conoscenze acquisite

7.4 Valutazione personale

Capitolo 8

Roba da organizzare

Suddivisione grafana in 3 database Cosa viene tracciato -> SQL (riconoscimento per repository e metodi), endpoint API (chiamate al server) Scelte progettuali fatte: sampling ratio, retention dati, componenti fondamentali da tracciare

Appendice A

Appendice A

Citazione

Autore della citazione

Acronimi e abbreviazioni

AI Artificial Intelligence. 8–10, 29

API Application Program Interface. 8, 9, 15, 16, 18, 29

CRM Customer Relationship Management. 1, 29

CRUD Create, Read, Update, Delete. vii, 22, 29

ERP Enterprise Resource Planning. 1, 29

LLM Large Language Model. 6, 7, 29

OTel OpenTelemetry. 4, 5, 18, 20

PoC Proof of Concept. 14, 29

RFID Radio-Frequency Identification. 1, 30

SDK Software Development Kit. 20, 30

SPA Single Page Application. 15, 30

UML Unified Modeling Language. 11, 30

Glossario

AI con *AI, Artificial Intelligence* si indica un insieme di tecniche e metodi informatici che consentono a un sistema di simulare capacità tipiche dell'intelligenza umana, quali apprendimento, ragionamento, percezione e risoluzione di problemi. 28

API in informatica con il termine *Application Programming Interface API* (ing. interfaccia di programmazione di un'applicazione) si indica ogni insieme di procedure disponibili al programmatore, di solito raggruppate a formare un set di strumenti specifici per l'espletamento di un determinato compito all'interno di un certo programma. La finalità è ottenere un'astrazione, di solito tra l'hardware e il programmatore o tra software a basso e quello ad alto livello semplificando così il lavoro di programmazione. 28

CRM con *CRM, Customer Relationship Management* si indica un sistema o insieme di strumenti software utilizzati per gestire le relazioni e le interazioni con clienti attuali e potenziali. Un CRM consente di organizzare e analizzare informazioni relative ai clienti, supportare le attività di vendita, marketing e assistenza, e migliorare la qualità del servizio e la fidelizzazione. 28

CRUD con *CRUD, Create, Read, Update, Delete* si indica l'insieme delle quattro operazioni fondamentali per la gestione dei dati in un sistema informatico. Rappresenta le azioni di creazione, lettura, aggiornamento ed eliminazione di entità all'interno di un database o di un'applicazione. Queste operazioni costituiscono la base per la maggior parte delle funzionalità di persistenza e manipolazione dei dati.. 28

ERP con *ERP, Enterprise Resource Planning* si indica un sistema informativo aziendale integrato utilizzato per gestire e automatizzare i principali processi operativi di un'organizzazione, quali contabilità, gestione delle risorse umane, produzione, logistica e vendite. 28

LLM con *LLM, Large Language Model* si indica un modello di intelligenza artificiale basato su reti neurali di grandi dimensioni, addestrato su ampie quantità di dati testuali, in grado di comprendere e generare linguaggio naturale. 28

PoC con *PoC, Proof of Concept* si indica un prototipo o implementazione preliminare sviluppata per verificare la fattibilità di una soluzione tecnica o di un'idea progettuale. Un PoC ha lo scopo di validare determinati aspetti, come prestazioni, integrazione o corretto funzionamento, prima di procedere con uno sviluppo completo. 28

RFID con *RFID, Radio-Frequency Identification* si indica una tecnologia che utilizza onde radio per identificare e tracciare oggetti, persone o animali attraverso dispositivi chiamati tag RFID. Questi tag contengono informazioni che possono essere lette a distanza da appositi lettori, senza necessità di contatto diretto o visibilità, rendendo la tecnologia particolarmente utile in ambiti come logistica, tracciamento e gestione degli inventari. 28

SCRUM metodologia agile per la gestione e lo sviluppo di progetti, basata su un approccio iterativo e incrementale. Il lavoro viene suddiviso in cicli temporali chiamati Sprint, al termine dei quali viene prodotto un incremento funzionante del sistema. Scrum definisce ruoli (come Scrum Master e Product Owner), eventi (Sprint, Daily Scrum, Sprint Review e Retrospective) e artefatti (Product Backlog e Sprint Backlog) per favorire collaborazione, adattabilità e miglioramento continuo del processo di sviluppo. 6

SDK con *SDK, Software Development Kit* si indica insieme di strumenti, librerie, documentazione e API messi a disposizione per facilitare lo sviluppo di applicazioni su una determinata piattaforma o tecnologia. Un SDK fornisce componenti riutilizzabili e funzionalità predefinite che permettono agli sviluppatori di integrare rapidamente servizi specifici, riducendo tempi e complessità di sviluppo.. 28

SPA con *SPA, Single Page Application* si indica un'applicazione web costituita da una singola pagina HTML che viene aggiornata dinamicamente senza ricaricare completamente il contenuto. Le SPA migliorano l'esperienza utente rendendo la navigazione più fluida e veloce, grazie al caricamento asincrono dei dati e alla gestione lato client della logica applicativa. 28

UML in ingegneria del software *UML, Unified Modeling Language* (ing. linguaggio di modellazione unificato) è un linguaggio di modellazione e specifica basato sul paradigma object-oriented. L'*UML* svolge un'importantissima funzione di "lingua franca" nella comunità della progettazione e programmazione a oggetti. Gran parte della letteratura di settore usa tale linguaggio per descrivere soluzioni analitiche e progettuali in modo sintetico e comprensibile a un vasto pubblico. 28

Bibliografia